P2P 네트워킹 (P2P Networking)

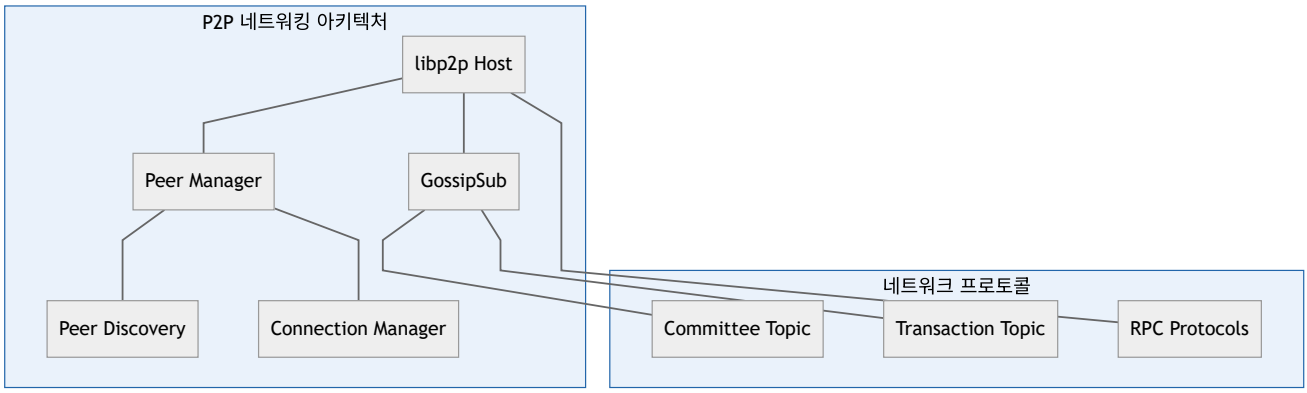
소개

Oasis Core는 libp2p 기반의 분산 P2P 네트워킹 시스템을 사용하여 노드 간 통신을 관리합니다. 이 시스템은 피어 발견, 연결 관리, 메시지 라우팅 및 전파를 담당하며, 블록체인 네트워크의 탈중앙화된 특성을 지원합니다.

아키텍처 개요

libp2p 기반 네트워킹

Oasis Core의 P2P 네트워킹은 다음과 같은 libp2p 컴포넌트들을 활용합니다:



주요 컴포넌트

1. P2P Host

- **역할**: libp2p 호스트 인스턴스 관리

- **기능**:
- 네트워크 연결 수락 및 관리
- 프로토콜 핸들러 등록
- 피어 식별 및 인증

2. GossipSub 라우터

- **역할**: 메시지 전파 및 구독 관리
- **기능**:
- 토픽 기반 메시지 라우팅
- 메시지 중복 제거
- 피어 간 메시지 전파

3. Peer Manager

- **역할**: 피어 연결 및 관리
- **기능**:
- 피어 발견 및 연결
- 연결 품질 관리
- 피어 태깅 및 우선순위 설정

피어 발견 메커니즘

1. Bootstrap Discovery

```
// Bootstrap 피어를 통한 초기 네트워크 진입
type BootstrapDiscoveryConfig struct {
```

```
Enable  bool

Seeds []peer.AddrInfo

RetentionPeriod time.Duration

}
```

동작 방식:

1. 설정된 시드 노드에 연결
2. 시드 노드로부터 다른 피어 정보 수집
3. 주기적으로 피어 정보 갱신

2. Registry-based Discovery

```
// 레지스트리를 통한 피어 발견

func (m *PeerManager) connectRegisteredPeers(ctx context.Context) {

// 레지스트리에서 등록된 피어 검색

for  p, d := range  m.protocols {

connected := m.NumProtocolPeers(p)

if  limit := d.min - connected;  limit > 0 {

peerCh := m.registry.findProtocolPeers(ctx, p)

m.connector.connectMany(ctx, peerCh, limit)

}

}

}
```

특징:

- 합의 레지스트리에 등록된 노드 정보 활용
- 신뢰할 수 있는 피어 우선 연결
- 프로토콜별 피어 요구사항 충족

3. DHT Discovery

```
// 분산 해시 테이블을 통한 피어 발견
func (d *peerDiscovery) findPeers(ctx context.Context, topic  string) <-chan pee
peerCh := make(chan peer.AddrInfo)
go  func() {
defer  close(peerCh)
// DHT를 통한 피어 검색
for  _, discovery := range  d.seeds {
peers, err := discovery.FindPeers(ctx, topic)
if  err != nil {
continue
}
for  peer := range  peers {
select {
case  peerCh <- peer:
case <-ctx.Done():
return
}
}
```

```
}
}()
return peerCh
}
```

연결 관리

Connection Gating

```
// 연결 제어 및 보안
type BasicConnectionGater struct {
blockedPeers map[peer.ID]bool
allowedPeers map[peer.ID]bool
}

func (g *BasicConnectionGater) InterceptPeerDial(p peer.ID) bool {
// 차단된 피어 확인
if g.blockedPeers[p] {
return false
}
return true
}
```

기능:

- 악의적인 피어 차단
- 허용된 피어 화이트리스트 관리
- 연결 수 제한 및 관리

Peer Tagging

```
// 피어 중요도 태깅
func (t *peerTagger) SetPeerImportance(kind ImportanceKind, runtimeID common.Nam
tag := fmt.Sprintf("%s:%s", kind, runtimeID.String())
for _, pid := range pids {
t.connMgr.TagPeer(pid, tag, importanceWeight[kind])
}
}
```

중요도 분류:

- **High**: 밸리데이터 및 핵심 노드
- **Medium**: 컴퓨트 워커 노드
- **Low**: 일반 클라이언트 노드

메시지 라우팅 및 전파

GossipSub 프로토콜

```
// GossipSub 설정
```

```
pubsub, err := pubsub.NewGossipSub(
ctx,
host,
pubsub.WithMessageSigning(true), // 메시지 서명 활성화
pubsub.WithStrictSignatureVerification(true), // 엄격한 서명 검증
pubsub.WithFloodPublish(true), // 플러드 발행 모드
pubsub.WithPeerExchange(true), // 피어 교환 활성화
pubsub.WithMessageIdFn(messageIdFn), // 메시지 ID 함수
pubsub.WithSeenMessagesTTL(seenMessagesTTL), // 중복 메시지 TTL
)
```

토픽 기반 라우팅

1. Committee Topic

```
// 위원회 메시지 토픽
func NewTopicKindCommitteeID(chainContext string, runtimeID common.Namespace)
return fmt.Sprintf("%s/committee/%s", chainContext, runtimeID.String())
}
```

용도:

- 배치 제안 전파
- 위원회 간 통신
- 합의 관련 메시지

2. Transaction Topic

```
// 트랜잭션 토픽
func NewTopicKindTxID(chainContext string, runtimeID common.Namespace) string
return fmt.Sprintf("%s/tx/%s", chainContext, runtimeID.String())
}
```

용도:

- 트랜잭션 전파
- 멤풀 동기화
- 사용자 제출 트랜잭션

메시지 검증

```
// 토픽 메시지 검증기
func (h *topicHandler) topicMessageValidator(ctx context.Context, peerID peer.ID
// 메시지 디코딩
decodedMsg, err := h.handler.DecodeMessage(msg.Data)
if err != nil {
return pubsub.ValidationReject
}
// 발신자 인증
senderPubKey := extractPeerPublicKey(peerID)
if err := h.handler.AuthorizeMessage(ctx, senderPubKey, decodedMsg); err != ni
```

```
return  pubsub.ValidationReject
}
return  pubsub.ValidationAccept
}
```

P2P 네트워킹 설정

기본 설정

```yaml
# P2P 네트워킹 설정 예제
p2p:
# 등록할 주소 목록
registration:
addresses:
- "192.168.1.100:9200"
# 연결 관리자 설정
connection_manager:
max_num_peers: 100
persistent_peers:
- "oasis1abc...@192.168.1.101:9200"
# GossipSub 설정
gossipsub:
peer_outbound_queue_size: 32
```

```yaml
validate_queue_size: 32

validate_throttle: 8192

# 피어 발견 설정

discovery:

bootstrap:

enable: true

retention_period: "1h"

# 시드 노드 목록

seeds:

- "oasis1seed1...@seed1.oasisprotocol.org:9200"

- "oasis1seed2...@seed2.oasisprotocol.org:9200"
```

고급 설정

```go
// 호스트 설정

type HostConfig struct {

Addresses []multiaddr.Multiaddr

Port  uint16

MaxNumPeers  int

PeerOutboundQueue  int

BlockedPeers []core.PeerID

}
```

```go
// 연결 관리 설정
type ConnectionManagerConfig struct {
MaxNumPeers int
PersistentPeers []peer.AddrInfo
LowWatermark int
HighWatermark int
}
```

사용 예제

1. P2P 서비스 초기화

```go
// P2P 서비스 생성
p2pService, err := p2p.New(identity, chainContext, store)
if err != nil {
return fmt.Errorf("P2P 서비스 생성 실패: %w", err)
}

// 서비스 시작
if err := p2pService.Start(); err != nil {
return fmt.Errorf("P2P 서비스 시작 실패: %w", err)
}
```

2. 메시지 핸들러 등록

```
// 메시지 핸들러 구현
type CommitteeHandler struct{}

func (h *CommitteeHandler) DecodeMessage(msg []byte) (any, error) {
var  proposal BatchProposal
return &proposal, cbor.Unmarshal(msg, &proposal)
}

func (h *CommitteeHandler) AuthorizeMessage(ctx context.Context, peerID signatur
// 발신자 권한 검증 로직
return  nil
}

func (h *CommitteeHandler) HandleMessage(ctx context.Context, peerID signature.P
// 메시지 처리 로직
return  nil
}

// 핸들러 등록
topic := protocol.NewTopicKindCommitteeID(chainContext, runtimeID)
```

```
p2pService.RegisterHandler(topic, &CommitteeHandler{})
```

3. 메시지 발행

```
// 메시지 발행
proposal := &BatchProposal{
Header: ProposalHeader{
Round: currentRound,
Timestamp: time.Now().Unix(),
},
Batch: transactions,
}

topic := protocol.NewTopicKindCommitteeID(chainContext, runtimeID)
p2pService.Publish(ctx, topic, proposal)
```

4. 프로토콜 서버 등록

```
// RPC 프로토콜 서버 구현
type StorageSyncServer struct {
protocol core.ProtocolID
}
```

```go
func (s *StorageSyncServer) Protocol() core.ProtocolID {
return  s.protocol
}

func (s *StorageSyncServer) HandleStream(stream network.Stream) {
// 스트림 처리 로직
defer  stream.Close()
// 요청 읽기 및 응답 전송
}

// 프로토콜 서버 등록
server := &StorageSyncServer{
protocol: "/oasis/storage/sync/1.0.0",
}
p2pService.RegisterProtocolServer(server)
```

모니터링 및 디버깅

상태 조회

```go
// P2P 상태 확인
status := p2pService.GetStatus()
fmt.Printf("연결된 피어 수: %d\n", status.NumPeers)
```

```go
fmt.Printf("연결 수: %d\n", status.NumConnections)


// 프로토콜별 피어 수
for protocol, count := range status.Protocols {
fmt.Printf("프로토콜 %s: %d 피어\n", protocol, count)
}


// 토픽별 피어 수
for topic, count := range status.Topics {
fmt.Printf("토픽 %s: %d 피어\n", topic, count)
}
```

피어 목록 조회

```go
// 런타임별 피어 목록
peers := p2pService.Peers(runtimeID)
for _, peer := range peers {
fmt.Printf("피어: %s\n", peer)
}
```

로깅 설정

```
# 로깅 설정
```

```yaml
log:
level: debug
format: JSON


# P2P 디버그 로깅 활성화
grpc:
log:
debug: true
```

보안 고려사항

1. 피어 인증

- 모든 메시지는 발신자 서명으로 검증
- 피어 ID는 공개키에서 파생
- TLS를 통한 전송 계층 보안

2. 메시지 검증

- 메시지 무결성 검증
- 중복 메시지 필터링
- 스팸 방지 메커니즘

3. 연결 제어

- 악의적인 피어 차단
- 연결 수 제한

- 대역폭 관리

성능 최적화

1. 연결 관리

```
// 연결 수 최적화
const (
minTopicPeers = 10  // 토픽당 최소 피어 수
totalTopicPeers = 20  // 토픽당 목표 피어 수
maxPeers = 100  // 최대 피어 수
)
```

2. 메시지 전파

```
// 메시지 전파 최적화
const (
seenMessagesTTL = 120 * time.Second  // 중복 메시지 TTL
maxMessageSize = 104857600  // 최대 메시지 크기 (100MB)
)
```

3. 대역폭 관리

- 피어별 대역폭 제한
- 우선순위 기반 메시지 전송

- 적응형 전파 전략

문제 해결

일반적인 문제

1. **피어 연결 실패**

- 네트워크 연결 확인
- 방화벽 설정 검토
- 시드 노드 상태 확인

2. **메시지 전파 지연**

- 피어 수 확인
- 네트워크 대역폭 점검
- GossipSub 설정 조정

3. **높은 메모리 사용량**

- 연결 수 제한 설정
- 메시지 TTL 조정
- 피어 정리 주기 단축

디버깅 도구

```
# P2P 상태 확인
oasis-node control status

# 피어 목록 조회
```

```
oasis-node debug p2p peers


# 연결 상태 확인
oasis-node debug p2p connections
```

이 문서는 Oasis Core의 P2P 네트워킹 시스템에 대한 포괄적인 가이드를 제공합니다. libp2p 기반의 분산 네트워킹 아키텍처를 통해 안전하고 효율적인 노드 간 통신을 구현할 수 있습니다.